



PROJEKTDOKUMENTATION

Entwicklung des Multiplayer-Würfelspiels „10K“

Android-App mit PHP-REST-API und MySQL-Datenbank

Ausbildungsberuf	Fachinformatiker/in – Fachrichtung Anwendungsentwicklung
Prüfung	IHK Berlin – Projektdokumentation
Prüfungsteilnehmer	Emanuel Kronas
Prüflingsnummer	
Bildungsträger	GPB Berlin-Neukölln
Praktikumsbetrieb	web nach mass
Projektzeitraum	15.06.2026 bis 09.07.2026
Projektumfang	54 Stunden

Dokumentationsstand: 10.07.2026 · Softwarestand: Version 1.1 · Git-Commit c29083e

Repository: github.com/Emu77/10k

Hinweis zur Verwendung

Diese Dokumentation wurde auf Grundlage des vollständigen Projektarchivs und des öffentlich verfügbaren GitHub-Repositories erstellt. Personenbezogene Prüfungsangaben, insbesondere die Prüflingsnummer, sind vor einer Einreichung zu ergänzen. Die Dokumentation beschreibt den tatsächlich implementierten Stand der Version 1.1 und kennzeichnet Abweichungen von früheren Planungsunterlagen transparent.

Das Projekt ist ein privates Lern- und Praxisprojekt, das während des Betriebspraktikums im Rahmen der Umschulung zum Fachinformatiker für Anwendungsentwicklung umgesetzt wurde. Es handelt sich nicht um ein kommerzielles Produkt und nicht automatisch um einen bereits genehmigten betrieblichen IHK-Projektantrag.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Projektdokumentation selbstständig angefertigt und nur die angegebenen Hilfsmittel verwendet habe. Alle sinngemäß oder wörtlich übernommenen Inhalte sind als solche kenntlich gemacht. Die im Projekt eingesetzten Quelltexte wurden von mir im Rahmen des beschriebenen Projekts erstellt beziehungsweise weiterentwickelt.

Ort, Datum: _____

Unterschrift: _____

Inhaltsverzeichnis

In LibreOffice/Word aktualisieren: Rechtsklick → Verzeichnis aktualisieren

Hinweis: Das automatische Inhaltsverzeichnis und die Seitenzahlen können nach dem Öffnen in LibreOffice Writer mit „Extras – Verzeichnisse aktualisieren“ beziehungsweise per Rechtsklick aktualisiert werden.

1 Projektübersicht

1.1 Ausgangssituation

Das Würfelspiel „Zehntausend“ wird traditionell mit fünf Würfeln, Papier und Stift gespielt. Punkte, Einstiegsbedingungen, Nullwürfe und Sonderwertungen müssen manuell kontrolliert werden. Ein gemeinsames Spiel über räumlich getrennte Android-Geräte ist in dieser Form nicht möglich. Ziel des Projekts war daher eine digitale Mehrspielerumsetzung, die die Spielregeln serverseitig prüft, den Spielstand zentral speichert und alle Teilnehmer über eine mobile Anwendung synchronisiert.

Die technische Ausgangslage bestand aus einer privaten Webhosting-Umgebung auf kronisoft.net mit Apache, PHP 7.4 und MySQL/MariaDB sowie einer Android-Entwicklungsumgebung. Eine bestehende Standardlösung sollte nicht angepasst werden; App, Backend, Datenmodell und Spiellogik wurden als eigenständiges System entwickelt.

1.2 Projektziel

Das Projektziel war die Entwicklung einer lauffähigen Android-App, über die zwei oder mehr Personen einen Spielraum erstellen, per sechsstelligen Code beitreten und eine vollständige Partie „10K“ spielen können. Optional sollten bis zu drei KI-Gegner teilnehmen. Die App sollte die Zielpunktzahl konfigurierbar machen, Würfe und gültige Auswahlen anzeigen, Punkte sichern, Nullwürfe und Hot Dice erkennen sowie eine abschließende Rangliste darstellen.

- Android-App in Kotlin mit Jetpack Compose und Material 3
- zentraler Spielzustand über eine PHP-REST-API
- persistent gespeicherte Räume, Spieler und Spielzüge in MySQL/MariaDB
- Token-basierte Zuordnung und Autorisierung der Spieler
- serverseitige Scoring-Engine als verbindliche Regelinstanz
- Online-Multiplayer durch regelmäßiges Polling des Spielzustands
- KI-Spieler mit einer nachvollziehbaren Greedy-Strategie
- Deployment auf der privaten Testumgebung kronisoft.net.

1.3 Projektabgrenzung

Nicht Bestandteil des Projekts waren Benutzerkonten mit E-Mail-Adresse und Passwort, ein öffentliches Matchmaking, In-App-Käufe, Push-Benachrichtigungen, eine iOS-Version, WebSockets, Chatfunktionen, manipulationssichere Zufallszahlen für Geldspiele sowie ein produktiver App-Store-Release. Das Webhosting und die bestehende Domain waren bereits vorhanden und wurden nicht im Projekt eingerichtet.

1.4 Projektumfeld und Beteiligte

Rolle / System	Bedeutung im Projekt
Projektbearbeiter	Planung, Konzeption, Entwicklung, Test, Deployment und Dokumentation
Praktikumsbetrieb „web nach mass“	Betrieblicher Kontext und fachliche Einordnung während des Praktikums
GPB Berlin-Neukölln	Bildungsträger der Umschulung
Spieler / Host	Erstellt einen Raum, konfiguriert das Spiel und startet die Partie
Mitspieler	Tritt per Raumcode bei und führt eigene Spielzüge aus
KI-Spieler	Vom Backend gesteuerter automatischer Teilnehmer

Rolle / System	Bedeutung im Projekt
kronisoft.net	Private Hosting- und Testumgebung für PHP-API und MySQL-Datenbank
GitHub, Repository Emu77/10k	Versionsverwaltung und nachvollziehbare Projekthistorie

2 Projektplanung

2.1 Vorgehensmodell

Für das Projekt wurde ein phasenorientiertes, iteratives Vorgehen gewählt. Die Grundstruktur orientierte sich an Analyse, Entwurf, Implementierung, Test und Übergabe. Innerhalb der Implementierung wurden Backend und Android-App in kleinen, lauffähigen Schritten weiterentwickelt. Fehler aus Integrationstests führten unmittelbar zu Korrektur-Commits. Dieses Vorgehen war zweckmäßig, weil das Datenmodell und die grundlegenden Spielregeln früh feststanden, während Bedienung, Mehrspieler-Synchronisation und Sonderregeln iterativ verfeinert werden mussten.

2.2 Zeitplanung

Phase	Arbeitspaket	Planzeit
1	Analyse und Planung	4 h
2	Datenbankentwicklung	4 h
3	Backend-Grundstruktur und Konfiguration	3 h
4	Backend-Spiellogik und API-Endpunkte	9 h
5	Android-Projektaufbau und Grundstruktur	8 h
6	Android-Lobby	6 h
7	Android-Spielfeld und Spielmechanik	10 h
8	Tests und Deployment	6 h
9	Dokumentation	4 h
	Gesamt	54 h

Der detaillierte Projektstruktur- und Zeitplan befindet sich in Anlage A. Die 54 Stunden beziehen sich auf den geplanten Kernumfang. Spätere Verbesserungen, die nach einem lauffähigen Grundstand aus Tests entstanden, werden im Soll-Ist-Vergleich ausgewiesen.

2.3 Ressourcen und Werkzeuge

Bereich	Eingesetzte Mittel
Entwicklungssystem	Ubuntu/Windows, Android Studio, Visual Studio Code
Android	Kotlin, Jetpack Compose, Material 3, Android SDK 35, Mindestversion API 26
Netzwerkzugriff	Retrofit 2, OkHttp, Gson, Kotlin Coroutines
Backend	PHP 7.4, Apache, PDO
Datenbank	MySQL/MariaDB, phpMyAdmin, InnoDB
Modellierung	draw.io / Graphviz
Versionierung	Git und GitHub
Deployment	FileZilla / Webhosting auf kronisoft.net
Tests	Android-Geräte, Emulator, Browser-/API-Tests, manuelle Spielszenarien

2.4 Risikoanalyse

Risiko	Auswirkung	Gegenmaßnahme
Abweichende PHP-Version auf dem Shared Hosting	API läuft lokal, aber nicht online	PHP-7.4-kompatible Syntax; produktionsnahe Tests
Instabile oder verzögerte Synchronisation	Spieler sehen veraltete Zustände	Zentraler Serverzustand; Polling; vollständiges Nachladen nach Aktionen
Ungültige Würfelauswahl	Falsche Punkte oder Manipulation	Jede Auswahl wird serverseitig validiert
Offengelegte Zugangsdaten	Datenbank kompromittiert	.gitignore; config.example.php statt echter Zugangsdaten
Unterschiedliche Displaygrößen	Würfel oder Bedienelemente werden abgeschnitten	Responsive Compose-Layouts; Safe-Area-Padding; variable Würfelbreite
Komplexität zusätzlicher Spielregeln	Zeitüberschreitung und Fehler	Priorisierung des Kernspiels; Erweiterungen erst nach lauffähigem Stand

3 Analyse und Fachkonzept

3.1 Funktionale Anforderungen

ID	Anforderung	Priorität
FA01	Spielraum mit eindeutigem sechsstelligen Code erstellen	MUSS
FA02	Per Raumcode beitreten; mindestens zwei Spieler vor dem Start	MUSS
FA03	0 bis 3 KI-Gegner und Zielwert 5.000/10.000/15.000 wählen	MUSS
FA04	Aktive Würfel werfen, wertbare Würfel auswählen und behalten	MUSS
FA05	Punkte sichern oder mit verbleibenden Würfeln weiterspielen	MUSS
FA06	Bust und Hot Dice automatisch erkennen	MUSS
FA07	Wertungsregeln für Einzelwürfel, Pasche, Straße und Full House	MUSS
FA08	Strichsystem: drei aufeinanderfolgende Nullrunden setzen Gesamtstand zurück	SOLL
FA09	Aktuellen Spieler, Punktestände, Würfel und Spielmeldungen synchron anzeigen	MUSS
FA10	Zielerreichung, Entscheidung zum Weiterspielen und Rangliste abbilden	SOLL

3.2 Nichtfunktionale Anforderungen

ID	Anforderung
NFA01	Die App muss auf Android ab API-Level 26 lauffähig sein.
NFA02	Der zentrale Spielzustand darf nicht ausschließlich auf einem Endgerät liegen.
NFA03	API-Antworten werden im JSON-Format übertragen und verständlich als Erfolg oder Fehler gekennzeichnet.
NFA04	Datenbankzugangsdaten dürfen nicht im öffentlichen Repository gespeichert werden.
NFA05	Die Bedienoberfläche soll auf typischen Smartphone-Breiten ohne horizontales Abschneiden nutzbar sein.
NFA06	Fehlerhafte Eingaben und nicht erlaubte Aktionen müssen serverseitig abgewiesen werden.
NFA07	Der Quellcode soll durch Schichten, Datenmodelle und zentrale Hilfsfunktionen wartbar bleiben.

3.3 Spielregeln als Geschäftslogik

Kombination	Punkte
Einzelne 1	100
Einzelne 5	50
Drei Einsen	1.000
Dreierpasch $n \neq 1$	$n \times 100$
Viererpasch	Dreierpasch $\times 2$
Fünferpasch	Dreierpasch $\times 4$
Straße 1-2-3-4-5	1.000
Full House aus drei und zwei gleichen Würfeln	1.000

Zusätzlich gelten Hot Dice, wenn alle fünf Würfel erfolgreich behalten wurden, sowie ein Nullwurf, wenn der aktuelle Wurf keine wertbare Auswahl enthält. Die aktuelle Regelquelle legt einen Mindesteinstieg von 300 Punkten fest. Das Strichsystem zählt Nullrunden nach erfolgtem Einstieg; drei aufeinanderfolgende Striche setzen den Gesamtpunktestand auf null zurück.

3.4 Use Cases

Use Case	Akteur	Kurzbeschreibung
UC01 Raum erstellen	Host	Name, KI-Anzahl und Zielpunktzahl senden; Raumcode und Token erhalten
UC02 Raum beitreten	Mitspieler	Code und Name senden; Spielerplatz und Token erhalten
UC03 Spiel starten	Host	Start ab mindestens zwei Teilnehmern auslösen
UC04 Würfeln	Aktiver Spieler	Noch aktive Würfel werfen; Bust automatisch prüfen
UC05 Würfel behalten	Aktiver Spieler	Auswahl serverseitig validieren und Zugpunkte erhöhen
UC06 Punkte sichern	Aktiver Spieler	Zugpunkte zum Gesamtstand addieren und Zug weitergeben
UC07 KI-Zug	System/KI	Beste wertbare Kombination auswählen und nach Strategie banken
UC08 Zielerreichung	Spieler/System	Rang vergeben und Entscheidung über Fortsetzung verarbeiten
UC09 Zustand synchronisieren	Alle Clients	Spielzustand regelmäßig abfragen und UI aktualisieren

4 Systementwurf

4.1 Architekturentscheidung

Die Anwendung wurde als Client-Server-System umgesetzt. Die Android-App ist für Darstellung und Benutzereingaben zuständig. Die verbindliche Spiellogik liegt im PHP-Backend. Dadurch können mehrere Geräte denselben Spielstand verwenden, und ein veränderter Client kann keine beliebigen Punkte direkt in die Datenbank schreiben. Die MySQL-Datenbank speichert langlebige Spiel-, Spieler- und Zugdaten.



Abbildung 1: Schichten und Kommunikationswege der Anwendung

Für die Synchronisation wurde Polling statt einer WebSocket-Verbindung gewählt. Die App fragt den Zustand regelmäßig ab. Diese Lösung ist auf einfachem Shared Hosting zuverlässig betreibbar und reduziert die Serverkomplexität. Der Nachteil ist zusätzlicher Netzwerkverkehr und eine kleine Aktualisierungsverzögerung, die für ein rundenbasiertes Würfelspiel akzeptabel ist.

4.2 Android-Architektur

Die Android-Anwendung trennt Benutzeroberfläche, Zustandsverwaltung und Datenzugriff. Compose-Screens senden Ereignisse an den GameViewModel. Dieser hält den UiState als StateFlow, startet Coroutines und ruft das Repository auf. Das Repository kapselt Retrofit und vereinheitlicht die Fehlerbehandlung. Datenklassen bilden die JSON-Antworten des Backends ab.

- LoginScreen: Name, KI-Anzahl, Zielpunktzahl und Raumcode
- LobbyScreen: Teilnehmerliste, Raumcode, Host-Start und Polling
- GameScreen: Würfel, Auswahl, Zugpunkte, Gesamtstände und Spielmeldungen
- ResultScreen: Rangfolge und Endstände
- GameViewModel: zentraler UI-Zustand und Benutzeraktionen
- GameRepository/ApiService: Netzwerkkommunikation und API-Vertrag

4.3 Ablauf einer Partie

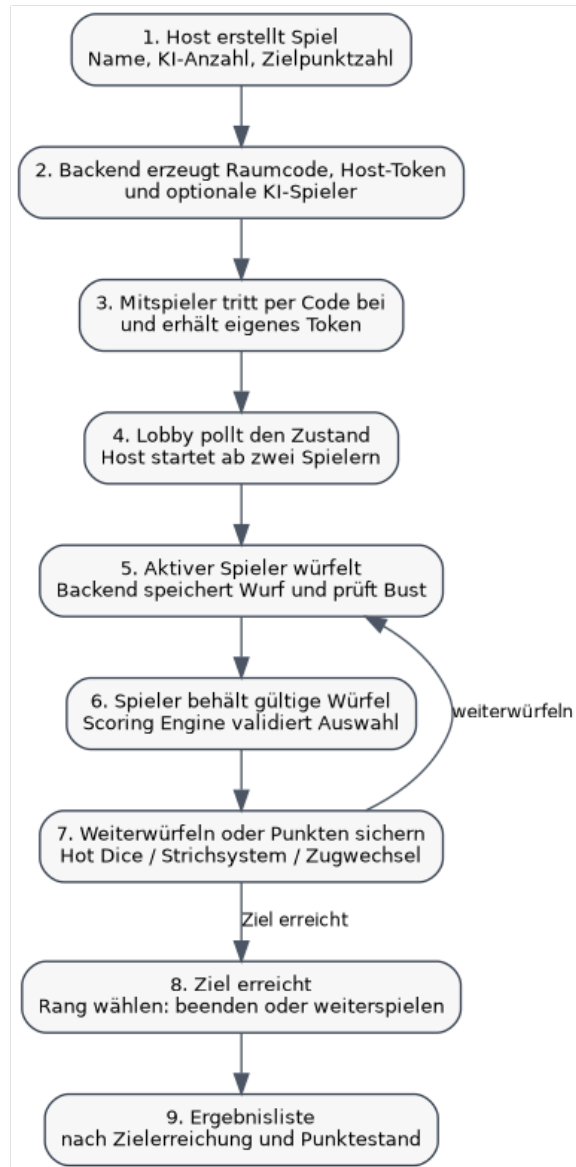


Abbildung 2: Vereinfachter Ablauf vom Erstellen bis zur Rangliste

4.4 Zustandsmodell

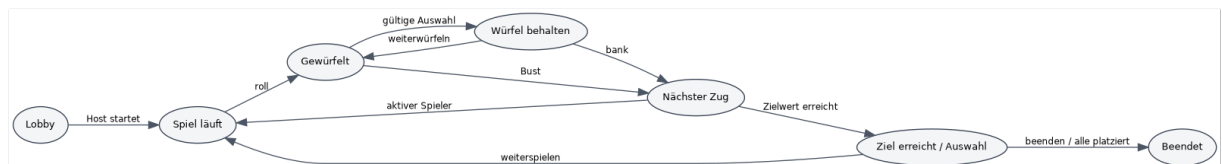


Abbildung 3: Wesentliche Zustandsübergänge

5 Datenbankentwurf

5.1 Datenmodell

Das relationale Modell besteht aus den Tabellen 10k_games, 10k_players und 10k_turns. Ein Spiel besitzt mehrere Spieler und mehrere protokollierte Spielzugaktionen. Spieler werden über game_id einem Spiel zugeordnet. Die Kombination aus game_id und slot ist eindeutig und legt die Reihenfolge fest. Beim Löschen eines Spiels werden abhängige Spieler über die Fremdschlüsselbeziehung automatisch entfernt.

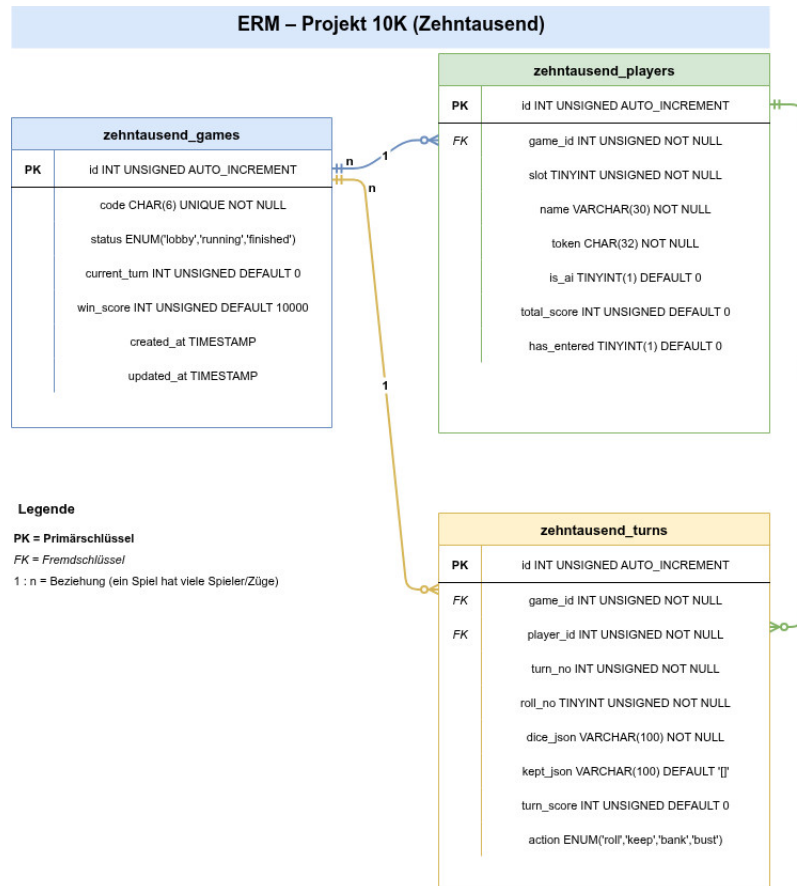


Abbildung 4: Entity-Relationship-Modell aus der Projektanalyse

5.2 Tabellenbeschreibung

Tabelle	Zweck	Wichtige Felder
10k_games	Raum und globaler Spielstatus	code, status, current_turn, win_score, created_at
10k_players	Teilnehmer und individuelle Werte	slot, name, token, is_ai, total_score, bust_streak, finish_rank, awaiting_choice
10k_turns	Nachvollziehbare Würfe und Aktionen	turn_no, roll_no, dice_json, kept_json, roll_score, turn_score, action

5.3 Begründung zentraler Datentypen

- CHAR(6) für den Raumcode, da seine Länge fest definiert ist.
- ENUM für Spielstatus und Zugaktion, um nur gültige Zustände zuzulassen.
- INT UNSIGNED für Punktestände und IDs, da negative Werte fachlich ausgeschlossen sind.

- CHAR(32) für das zufällig erzeugte Sitzungstoken.
- JSON-ähnliche Serialisierung in VARCHAR(300) für Würfelzustände, weil PHP 7.4 und das Hosting einfach unterstützt werden mussten.
- InnoDB wegen Transaktions- und Fremdschlüsselunterstützung.

5.4 Datenschutz und Sicherheit

Gespeichert werden nur frei gewählte Spielernamen, Spielstände und technische Tokens. Es werden keine E-Mail-Adressen, Passwörter oder Zahlungsdaten verarbeitet. Die Datenbankkonfiguration ist durch .gitignore vom Repository ausgeschlossen; config.example.php dokumentiert lediglich die benötigten Variablen. Jede spielerbezogene Aktion prüft Token, Spiel und aktiven Spieler im Backend. Die REST-Kommunikation erfolgt in der Produktionsumgebung über HTTPS.

6 Implementierung

6.1 Backend-Grundstruktur

Die Datei `api/index.php` dient als zentraler Einstiegspunkt. Der Parameter `action` wird gegen eine Whitelist geprüft und anschließend an die passende PHP-Datei weitergeleitet. Dadurch verwendet die Android-App eine einheitliche Basis-URL, während die einzelnen Anwendungsfälle getrennt implementiert bleiben.

```
<?php
require_once __DIR__ . '/_helpers.php';

$action = $_GET['action'] ?? $_POST['action'] ?? '';

$allowed = ['create', 'join', 'start', 'roll', 'keep', 'bank', 'state', 'ai_turn',
'finish_choice'];

if (!in_array($action, $allowed)) {
    http_response_code(400);
    echo json_encode(['success' => false, 'message' => 'Ungültige Aktion']);
    exit;
}

require_once __DIR__ . '/' . $action . '.php';
```

Quelltext 1: Zentrale Weiterleitung der erlaubten API-Aktionen

6.2 Scoring-Engine

Die Klasse `Scoring` berechnet mögliche Wertungen und validiert eine vom Client gesendete Auswahl erneut gegen die tatsächlich verfügbaren Würfel. Die Validierung ist bewusst serverseitig. Ein manipulierter Client kann dadurch keine nicht vorhandenen Würfel oder ungültige Kombinationen einreichen. Für kombinierte Wertungen, beispielsweise Dreierpasch plus einzelne Eins, wird die Auswahl in gültige Gruppen zerlegt und aufsummiert.

```
public static function validateSelection(array $selected, array $available) {
    if (empty($selected)) return false;
    if (!self::subsetAvailable($available, $selected)) return false;

    if (count($available) === 5 && count($selected) === 5) {
        $sortedSel = $selected; sort($sortedSel);
        $sortedAvail = $available; sort($sortedAvail);
        if ($sortedSel === $sortedAvail) {
            if ($sortedSel === [1,2,3,4,5]) return 1000;
            $vals = array_unique($sortedSel);
            if (count($vals) === 2) {
                $cnts = array_count_values($sortedSel);
                $c = array_values($cnts);
                sort($c);
                if ($c === [2, 3]) return 1000;
            }
        }
    }

    return self::scoreSelection($selected);
}
```

Quelltext 2: Serverseitige Validierung einer Würfelauswahl (gekürzt)

6.3 API-Endpunkte

Aktion	Methode	Aufgabe
create	POST	Spiel, Host und optionale KI-Spieler anlegen

Aktion	Methode	Aufgabe
join	POST	Spielraum prüfen und freien Spielerplatz anlegen
start	POST	Host prüfen und Spielstatus auf running setzen
roll	POST	Würfeln, Zustand speichern und Nullwurf erkennen
keep	POST	Auswahl validieren und Zugpunkte erhöhen
bank	POST	Punkte sichern, Strichfolge zurücksetzen und Zug wechseln
state	POST	Vollständigen Spielzustand für einen Client liefern
ai_turn	POST	Einen oder mehrere KI-Züge automatisch ausführen
finish_choice	POST	Entscheidung nach Zielerreichung verarbeiten

6.4 Mehrspieler-Synchronisation

Nach dem Erstellen oder Beitreten speichert der ViewModel Spiel-ID, Spieler-ID, Namen, Raumcode und Token im UiState. In Lobby und Spiel wird state regelmäßig aufgerufen. Nach zustandsändernden Aktionen lädt syncFullState den vollständigen Zustand erneut. Dieses Vorgehen verhindert, dass Teilantworten von roll, keep oder bank zu einem unvollständigen UI-Zustand führen.

```
private suspend fun syncFullState() {
    val state = _uiState.value
    repository.getState(state.gameId, state.playerId, state.token)
        .onSuccess { _uiState.value = _uiState.value.copy(gameState = it) }
        .onFailure { _uiState.value = _uiState.value.copy(error = it.message) }
}
```

Quelltext 3: Zentrales Nachladen des vollständigen Spielzustands

Ein wiederkehrendes Testproblem war, dass der Wechsel zur Ergebnisanzeige ursprünglich ausschließlich an die eigene Bank-Aktion gekoppelt war. Beendete stattdessen ein KI-Gegner die Partie im Hintergrund, etwa im Rahmen des Fast-Forward-Mechanismus, verblieb ein zuschauender Spieler auf einem eingefrorenen Spielbildschirm, obwohl das Backend den Spielstatus bereits auf finished gesetzt hatte. Der GameScreen prüft deshalb den Spielstatus bei jeder aktualisierten Zustandsabfrage und löst den Wechsel zur Ergebnisanzeige unabhängig davon aus, welcher Spieler das Spielende herbeigeführt hat.

6.5 Benutzeroberfläche

Die Oberfläche wurde deklarativ mit Jetpack Compose umgesetzt. Alle Screens verwenden ein dunkles Material-3-Theme. Eingabeschaltflächen werden deaktiviert, wenn Pflichtangaben fehlen oder eine Aktion fachlich nicht erlaubt ist. Safe-Drawing-Padding schützt Inhalte vor Status- und Navigationsleiste. Die Würfelreihe berechnet ihre Breite anhand von fünf Slots, sodass auf unterschiedlichen Displaybreiten keine Würfel abgeschnitten werden.

Screen	Wesentliche Funktionen
Login	Spielernamen merken, Spiel erstellen oder beitreten, KI-Anzahl und Zielwert wählen
Lobby	Raumcode anzeigen, Teilnehmer pollend aktualisieren, Start nur für Host und ab zwei Spielern

Screen	Wesentliche Funktionen
Spiel	Aktiven Spieler hervorheben, würfeln, Würfel markieren, behalten, banken, Statusmeldungen anzeigen
Ergebnis	Medaillen-/Rangfolge und Punktestände sortiert darstellen

Zusätzlich steht auf dem Spiel-Screen dauerhaft eine Schaltfläche „Spiel verlassen“ zur Verfügung. Sie setzt den lokalen App-Zustand zurück und führt unabhängig vom aktuellen Serverstatus zur Startseite zurück. Diese Funktion dient als Rückfalloption für Testsituationen, in denen die automatische Synchronisation ausnahmsweise nicht wie erwartet reagiert, und wurde nach entsprechenden Testbeobachtungen ergänzt.

6.6 KI-Strategie

Die KI verwendet eine Greedy-Strategie: Aus allen aktuell möglichen Wertungen wählt sie die Option mit dem höchsten unmittelbaren Punktwert. Ab einem definierten Zugwert sichert sie Punkte; andernfalls würfelt sie weiter. Nach Einführung des erweiterten Spielendes verarbeitet ein einzelner Aufruf von `ai_turn` automatisch mehrere KI-Züge nacheinander (begrenzt auf maximal 500 Züge beziehungsweise 20 Sekunden Laufzeit als Sicherheitsgrenze für das Shared Hosting), sobald kein menschlicher Spieler mehr auf eine Entscheidung wartet. Dadurch muss ein zuschauender Spieler nicht auf mehrere, durch das zweisekündige Client-Polling verzögerte Einzelanfragen warten, bis das Spiel zu Ende gespielt ist. Die Strategie ist bewusst nachvollziehbar und deterministisch in ihrer Auswahl, ohne komplexe Wahrscheinlichkeitsbewertung.

7 Qualitätssicherung und Tests

7.1 Teststrategie

Die Qualitätssicherung erfolgte auf mehreren Ebenen: isolierte Prüfung der Wertungslogik, manuelle API-Aufrufe, Integrationstests zwischen Android-App und Backend sowie vollständige Spielszenarien mit mehreren Geräten beziehungsweise KI-Spielern. Die Git-Historie dokumentiert Fehlerkorrekturen aus diesen Tests, beispielsweise bei Feldnamen, Token-Übergabe, Lobby-Start, Würfelloyout und Zustandsnachladung.

7.2 Ausgewählte Testfälle

ID	Testfall	Erwartetes Ergebnis	Ergebnis
T01	Spiel mit Name, 2 KI und Ziel 5.000 erstellen	Code, Token, Host und KI-Spieler werden angelegt	bestanden
T02	Mit ungültigem oder unbekanntem Code beitreten	Fehlermeldung; kein Spieler wird angelegt	bestanden
T03	Start mit nur einem Teilnehmer	Startschaltfläche deaktiviert / Backend lehnt ab	bestanden
T04	Wurf 1-1-1-5-3; 1-1-1-5 auswählen	1.050 Punkte werden erkannt	bestanden
T05	Straße 1-2-3-4-5 komplett auswählen	1.000 Punkte	bestanden
T06	Full House komplett auswählen	1.000 Punkte	bestanden
T07	Wurf ohne wertbare Kombination	Bust, Zugpunkte verfallen, nächster Spieler	bestanden
T08	Alle fünf Würfel behalten	Hot Dice; erneuter Wurf mit fünf Würfeln	bestanden
T09	Drei aufeinanderfolgende Busts nach Einstieg	Gesamtpunktestand wird auf 0 gesetzt	bestanden
T10	Nicht aktiver Spieler sendet roll	Backend weist Aktion zurück	bestanden
T11	Zielwert wird erreicht und Spieler wählt Weiterspielen	finish_rank wird gesetzt; Partie läuft weiter	bestanden
T12	App auf schmalen Display	Fünf Würfel passen nebeneinander; keine Überdeckung	bestanden
T13	KI gewinnt im Hintergrund, nachdem ein Mensch bereits weiterspielen liess	Ergebnisanzeige erscheint automatisch ohne Aktion des zuschauenden Spielers	bestanden
T14	„Spiel verlassen“ während laufender Partie betätigen	Rückkehr zur Startseite unabhaengig vom Serverstatus jederzeit möglich	bestanden

7.3 Fehlerbehandlung

Die API gibt bei ungültigen Eingaben einen Fehlerstatus und eine verständliche Meldung zurück. Im Repository werden nicht erfolgreiche HTTP-Antworten und Exceptions in Result.failure überführt. Der ViewModel schreibt die Meldung in den UiState. Automatische KI-Hintergrundprüfungen unterdrücken dagegen kurzzeitige Fehler bewusst, damit sie die Bedienung eines menschlichen Spielers nicht mit technisch irrelevanten Meldungen unterbrechen.

7.4 Deployment und Konfiguration

1. Datenbanktabellen mit setup.sql in phpMyAdmin anlegen.
2. config.php anhand von config.example.php mit produktiven Zugangsdaten erstellen.
3. Backend-Dateien in das Projektverzeichnis auf kronisoft.net übertragen.
4. Basis-URL im RetrofitClient auf das produktive Backend setzen.
5. Release-Build der Android-App erzeugen und auf Testgeräten installieren.
6. Mehrspieler-Szenario mit mindestens zwei Clients prüfen.

Eine Besonderheit des Hostings war, dass bestimmte .htaccess-Direktiven zu HTTP-500-Fehlern führten. Die Datei wurde deshalb auf eine mit der Umgebung kompatible Minimalversion reduziert. Außerdem musste der PHP-Code an PHP 7.4 angepasst werden. Ab Version 1.1 werden Release-APKs nicht mehr als Binärdateien im Repository versioniert, sondern ausschließlich als Anhänge zu den jeweiligen GitHub-Releases bereitgestellt; ein lokaler Ausgabeordner dient lediglich als Zwischenablage vor dem Hochladen. Dadurch bleibt die Repository-Größe unabhängig von der Anzahl der Testbuilds gering.

8 Projektverlauf und Soll-Ist-Vergleich

8.1 Wesentliche Änderungen gegenüber der frühen Planung

Planungsstand	Tatsächliche Umsetzung	Begründung
Acht API-Endpunkte	Neun Aktionen einschließlich finish_choice	Erweitertes Spielende mit Entscheidung zum Weiterspielen
Mindesteinstieg teilweise 1.000 Punkte bzw. deaktiviert	Aktuelle Regelquelle: 300 Punkte	Regeln wurden fachlich vereinheitlicht; ältere Dokumente blieben als Historie erhalten
Grundwertungen und Hot Dice	Zusätzlich Straße, Full House und Strichsystem	Anpassung an die gewünschte Spielvariante
Spiel endet beim ersten Zielerreicher	Ränge und freiwilliges Weiterspielen	Bessere Mehrspieler-Erfahrung und vollständige Ergebnisliste
Einfache Würfeldarstellung	Responsive fünf-Slot-Berechnung	Tests auf realen Smartphone-Breiten zeigten Layoutprobleme
Getrennte Lobby-/Game-ViewModels im ersten Entwurf	Zentraler GameViewModel und Repository	Vereinfachte Zustandsweitergabe und Fehlerbehandlung
Ergebnisanzeige nur bei eigener Bank-Aktion ausgelöst	Automatischer Wechsel zur Ergebnisanzeige unabhängig vom auslösenden Spieler; zusätzlicher Notausgang „Spiel verlassen“	Testszenarios zeigten, dass zuschauende Spieler sonst auf einem eingefrorenen Bildschirm verbleiben, wenn ein KI-Gegner im Hintergrund gewinnt

8.2 Soll-Ist-Bewertung

Die Muss-Ziele des Kernsystems wurden erreicht: Räume können erstellt und betreten werden, der Host startet eine Partie, Spielzüge werden serverseitig validiert, Punktestände werden zentral synchronisiert und KI-Gegner können teilnehmen. Die zusätzlichen Regeln und die Rangfortsetzung erweitern den ursprünglichen Umfang. Der größte Mehraufwand entstand nicht in der Grundarchitektur, sondern in Integration und Bedienbarkeit: Tokenübergabe, Feldnamen zwischen Client und API, vollständige Zustandsaktualisierung und responsive Darstellung mussten iterativ korrigiert werden.

Kriterium	Bewertung
Funktionaler Umfang	erfüllt und um Rangfortsetzung/Strichsystem erweitert
Technische Architektur	Client-Server-Trennung und serverseitige Validierung umgesetzt
Datenhaltung	relationales Modell mit Fremdschlüsseln und Zugprotokoll umgesetzt
Sicherheit	Konfiguration aus Repository entfernt; tokenbasierte Aktionsprüfung
Bedienbarkeit	Dark Theme, klare Screens, deaktivierte ungültige Aktionen, responsive Würfel
Wartbarkeit	Schichtenmodell, zentrale Scoring-Klasse, einheitlicher API-Einstieg
Offene Punkte	automatisierte Unit-/UI-Tests, WebSockets, Bereinigung alter Spieldaten

9 Wirtschaftlichkeit

9.1 Kostenbetrachtung

Da es sich um ein privates Lernprojekt handelt, stehen Kompetenzaufbau und technische Machbarkeit im Vordergrund. Die vorhandene Domain, das Webhosting und die Entwicklungsgeräte verursachen für das Projekt keine zusätzlichen Anschaffungskosten. Zur nachvollziehbaren Bewertung wird dennoch ein kalkulatorischer Stundensatz angesetzt.

Kostenposition	Ansatz	Betrag
Projektarbeit	54 h × 15,00 € kalkulatorisch	810,00 €
Hosting/Domain	vorhandene private Infrastruktur; anteilig	0,00 € zusätzlich
Softwarelizenzen	Android Studio, Kotlin, PHP, MySQL/MariaDB, Git	0,00 €
Testgeräte	vorhanden	0,00 € zusätzlich
Gesamt		810,00 € kalkulatorisch

9.2 Nutzwert

Der Nutzen liegt in einer unmittelbar spielbaren Anwendung und vor allem in der praktischen Verbindung mehrerer Ausbildungsinhalte: mobile UI-Entwicklung, REST-Kommunikation, relationale Datenbanken, serverseitige Geschäftslogik, Authentisierung, Deployment und Versionsverwaltung. Gegenüber einer rein lokalen App demonstriert das Projekt zusätzlich reale Integrationsprobleme und deren strukturierte Behebung.

9.3 Make-or-Buy-Betrachtung

Für den reinen Spielzweck wären vorhandene Würfel-Apps wirtschaftlich günstiger. Eine Fremdlösung hätte jedoch die individuellen Regeln, die private Serverumgebung und den beabsichtigten Lerneffekt nicht abgedeckt. Die Eigenentwicklung war daher sachgerecht, weil der Projekterfolg nicht allein an einer kostengünstigen Nutzung, sondern am Nachweis der Entwicklungskompetenz gemessen wird.

10 Fazit und Ausblick

Mit „10K“ wurde ein vollständiges Client-Server-System von der Regelanalyse bis zur installierbaren Android-App umgesetzt. Die serverseitige Scoring-Engine bildet die zentrale fachliche Komponente. Durch die Trennung von Compose-Oberfläche, ViewModel, Repository, REST-API und Datenbank bleibt das System nachvollziehbar und erweiterbar. Die Mehrspielerfunktion arbeitet auf einfachem Shared Hosting ohne zusätzliche Serverdienste.

Besonders lehrreich waren die Schnittstellen zwischen den Komponenten. Mehrere Fehler entstanden nicht innerhalb einer einzelnen Methode, sondern durch abweichende Feldnamen, unvollständige Antworten oder fehlende Tokens zwischen App und API. Die iterative Fehlerbehebung und die nachvollziehbare Git-Historie zeigen, dass Integrationstests für verteilte Anwendungen ebenso wichtig sind wie die Implementierung der einzelnen Komponenten.

Als nächste Schritte bieten sich automatisierte Tests der Scoring-Klasse, instrumentierte Android-UI-Tests, serverseitige Transaktionen für komplexe Zugwechsel, Rate-Limiting, Ablauf und Bereinigung alter Räume sowie optional eine WebSocket-basierte Synchronisation an. Für einen öffentlichen Release wären außerdem Datenschutzinformationen, signierte Release-Builds und ein abgesichertes Betriebs- und Monitoringkonzept erforderlich.

Glossar

Begriff	Erklärung
API	Programmierschnittstelle zwischen Android-App und Backend
Bust / Nullwurf	Wurf ohne wertbare Auswahl; die Zuggpunkte verfallen
Hot Dice	Alle fünf Würfel wurden gewertet; es darf mit allen fünf weitergewürfelt werden
Jetpack Compose	Deklaratives UI-Framework für Android
KI	Künstlicher Mitspieler, der Entscheidungen automatisch ausführt
MVVM	Architekturmuster mit View, ViewModel und Model/Datenzugriff
Polling	Regelmäßiges Abfragen eines Serverzustands
REST	Architekturstil für HTTP-basierte Schnittstellen
StateFlow	Reaktiver Datenstrom für den UI-Zustand in Kotlin
Token	Zufällige Zeichenfolge zur Zuordnung und Autorisierung eines Spielers

Quellen- und Hilfsmittelverzeichnis

Quelle / Hilfsmittel	Verwendung
Projektarchiv 10k.zip, Stand 09.07.2026	Quellcode, Git-Historie, Anforderungen, API-Spezifikation, ER-Modell und Spielregeln
GitHub Repository Emu77/10k, geprüft am 10.07.2026	Abgleich des veröffentlichten Projektstands und README
Android Developer Documentation	Grundlagen zu Jetpack Compose, Navigation und Android-App-Konfiguration
Kotlin Coroutines / StateFlow Documentation	Asynchrone Abläufe und Zustandsverwaltung
Retrofit / OkHttp Documentation	HTTP-Kommunikation und JSON-Konvertierung
PHP Documentation	PHP-7.4-kompatible Implementierung, PDO und JSON
MySQL/MariaDB Documentation	Datentypen, Fremdschlüssel und InnoDB

Bei der Erstellung und sprachlichen Überarbeitung der Dokumentation wurde ein KI-gestütztes Textwerkzeug als Hilfsmittel eingesetzt. Fachliche Aussagen, Projekthinhalte und Quelltextbezüge wurden anhand des tatsächlichen Projektstands geprüft.

Anlage A – Detaillierter Projektstrukturplan

PSP-ID	Arbeitspaket	Zeit
1.1	Spielregeln und Anforderungen analysieren	1,0 h
1.2	Use Cases und Abgrenzung erstellen	1,0 h
1.3	Datenmodell und API entwerfen	1,5 h
1.4	Projektablauf und Risiken planen	0,5 h
2.1	Lokale Datenbank und Tabellen erstellen	2,0 h
2.2	setup.sql und Produktionsimport	1,5 h
2.3	Verbindung und Beziehungen prüfen	0,5 h
3.1	Backendstruktur, Konfiguration und Helper	2,0 h
3.2	CORS, Routing und Fehlerformat	1,0 h
4.1	Scoring-Engine implementieren	2,0 h
4.2	Raumverwaltung create/join/start	2,0 h
4.3	Spielaktionen roll/keep/bank	3,0 h
4.4	Status und KI-Endpunkt	1,5 h
4.5	Backend-Integrationstest	0,5 h
5.1	Android-Projekt und Dependencies	1,5 h
5.2	Datenmodelle, Retrofit und Repository	2,5 h
5.3	ViewModel und Zustandsmodell	2,0 h
5.4	Theme, Navigation und Grundscreens	2,0 h
6.1	Erstellen-/Beitreten-Oberfläche	2,5 h
6.2	Lobby, Teilnehmerliste und Polling	2,5 h
6.3	Host-Start und Navigation	1,0 h
7.1	Spielfeld und Würfeldarstellung	3,0 h
7.2	Auswahl, Keep und Bank	3,0 h
7.3	Statusmeldungen, KI und Ergebnis	2,5 h
7.4	Responsive Anpassungen	1,5 h
8.1	Fachliche Testfälle	2,0 h
8.2	Mehrgeräte-/Integrationstests	2,0 h
8.3	Deployment und Release-Build	2,0 h
9.1	Projektdokumentation	3,0 h
9.2	Anlagen und Abschlussprüfung	1,0 h

Anlage B – API-Schnittstelle

Produktive Basis-URL laut aktuellem Android-Quellcode:

<https://kronisoft.net/projekte/10k/backend/>

Aufrufschema: `api/index.php?action=<aktion>`

Aktion	Eingaben (Auswahl)	Rückgabe (Auswahl)	Authentisierung
create	name, ai_count, win_score	game_id, player_id, code, token	nein
join	code, name	game_id, player_id, code, token	nein
start	game_id, player_id, token	Erfolg / Meldung	Host-Token
roll	game_id, player_id, token	Würfel-/Aktionsstatus	Spieler-Token
keep	game_id, player_id, selected, token	Zugstatus	Spieler-Token
bank	game_id, player_id, token	Erfolg / Meldung	Spieler-Token
state	game_id, player_id, token	vollständiger GameState	Spieler-Token
ai_turn	game_id	aktualisierter GameState	serverintern gesteuert
finish_choice	token, choice	Erfolg / Meldung	Spieler-Token

Anlage C – Datenbankdefinition (Auszug)

```
-- 10K Würfelspiel – Datenbankstruktur
-- Einmal ausführen (z.B. via phpMyAdmin)

CREATE TABLE IF NOT EXISTS `10k_games` (
  `id`          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `code`        CHAR(6) NOT NULL UNIQUE,
  `status`      ENUM('lobby','running','finished') NOT NULL DEFAULT 'lobby',
  `current_turn` INT UNSIGNED NOT NULL DEFAULT 0, -- index in players-Reihenfolge
  `win_score`   INT UNSIGNED NOT NULL DEFAULT 10000,
  `created_at`  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  `updated_at`  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `10k_players` (
  `id`          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `game_id`     INT UNSIGNED NOT NULL,
  `slot`       TINYINT UNSIGNED NOT NULL, -- 0-based Reihenfolge
  `name`       VARCHAR(30) NOT NULL,
  `token`      CHAR(32) NOT NULL, -- session-Token für Auth
  `is_ai`      TINYINT(1) NOT NULL DEFAULT 0,
  `total_score` INT UNSIGNED NOT NULL DEFAULT 0,
  `has_entered` TINYINT(1) NOT NULL DEFAULT 0, -- mind. 1x ≥ 1000 gepunktet
  `bust_streak` TINYINT UNSIGNED NOT NULL DEFAULT 0,
  `finish_rank` TINYINT UNSIGNED NOT NULL DEFAULT NULL, -- Reihenfolge des Zielerreichens
  (1=Gold, 2=Silber, 3=Bronze)
  `awaiting_choice` TINYINT(1) NOT NULL DEFAULT 0, -- wartet auf Weiterspielen/Beenden-
  Entscheidung
  FOREIGN KEY (`game_id`) REFERENCES `10k_games`(`id`) ON DELETE CASCADE,
  UNIQUE KEY `game_slot` (`game_id`, `slot`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `10k_turns` (
  `id`          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `game_id`     INT UNSIGNED NOT NULL,
  `player_id`   INT UNSIGNED NOT NULL,
  `turn_no`     INT UNSIGNED NOT NULL,
  `roll_no`     TINYINT UNSIGNED NOT NULL,
  `dice_json`   VARCHAR(300) NOT NULL, -- alle 6 Würfel als JSON
  [{"v":3,"kept":false},...]
  `kept_json`   VARCHAR(300) NOT NULL DEFAULT '[]', -- behaltene Würfel
  `roll_score`  INT UNSIGNED NOT NULL DEFAULT 0,
  `turn_score`  INT UNSIGNED NOT NULL DEFAULT 0,
  `action`     ENUM('roll','keep','bank','bust') NOT NULL,
  `created_at`  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (`game_id`) REFERENCES `10k_games`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Anlage D – Repository- und Versionsstand

Für die Dokumentation wurde der lokale Git-Stand des hochgeladenen Archivs ausgewertet und mit der öffentlich sichtbaren Repository-Beschreibung abgeglichen.

Merkmal	Stand
Repository	github.com/Emu77/10k
Branch	main
Dokumentierter Commit	c29083e
Versions-Tag	v1.1.0
App-Version	1.1, versionCode 2
Letztes Datum im ausgewerteten Archiv	09.07.2026
Quellcodeumfang	ca. 2.126 Zeilen Kotlin, PHP und SQL ohne Builddateien
Programmiersprachen	Kotlin und PHP
Hauptverzeichnisse	app, backend, docs

Ausgewählte Entwicklungsschritte:

Zeitraum	Entwicklungsschwerpunkt
15.06.2026	Analyseunterlagen, Datenmodell, API-Spezifikation und Backend-Grundstand
17.–19.06.2026	Android-App, KI-Aufrufe, Tokenübergabe und erste Integrationskorrekturen
27.–28.06.2026	Straße, Full House, Strichsystem und Sicherheitsbereinigung der Konfiguration
01.–05.07.2026	Datenmodelle, kombinierte Wertungen, Dark Theme und vollständige App-Synchronisation
06.–07.07.2026	Lobby-, Start-, Token- und Meldungsfehler behoben; Bedienung verbessert
08.–09.07.2026	Responsive Würfel, Rangfortsetzung, Ergebnisdarstellung und Version 1.1
09.–10.07.2026	KI-Fast-Forward nach Zielerreichung, automatischer Wechsel zur Ergebnisanzeige, Notausgang „Spiel verlassen“, Ergebnisliste nach Zielerreichungsreihenfolge sortiert, Spielernamen wird vorgeschlagen, GitHub-Release v1.1.0 und Bereinigung der Repository-Struktur

Anlage E – Abnahmecheckliste

Prüfpunkt	Status
Projekt lässt sich in Android Studio öffnen und synchronisieren	☐
Backend-Konfiguration ist außerhalb des Repositories gesetzt	☐
Datenbankstruktur wurde importiert	☐
Spiel kann erstellt und per Code betreten werden	☐
Start ist erst ab zwei Spielern möglich	☐
Wertungen, Bust, Hot Dice und Strichsystem geprüft	☐
KI-Spieler führen vollständige Züge aus	☐
Zielerreichung und Rangliste geprüft	☐
Release-APK auf mindestens einem realen Gerät installiert	☐
Inhaltsverzeichnis, Prüflingsnummer und Seitenzahlen aktualisiert	☐